INDEPENDENT REFERENCE DESIGN

Reliable AWS Data Pipelines

Architecture decisions and failure handling

Sachin Bose Pulikkathara | Data & Cloud Architecture

The design problem

Turn incoming operational records into trusted analytical tables while making failures visible, recoverable, and safe to replay. This is a synthetic reference architecture, not a deployed system.

Proposed flow

S3 raw landing zone > AWS Glue transformation > quality checks > Redshift staging > validate and publish. Invalid records enter a quarantine path. Step Functions coordinates execution and recovery.

Make replay safe

Assign a batch identifier and retain an input reference. Define business keys and version rules, deduplicate consistently, and validate in staging before an appropriate transactional update/insert strategy.

Treat quality as a publication decision

Check required fields, types, allowable values, key uniqueness, and freshness. Define row-level quarantine rules and the failures that stop the entire batch.

Recover deliberately

Use bounded retries and backoff for transient failures. Route deterministic validation failures to review. Capture accepted, rejected, duplicate, and published counts alongside quality results and execution status.

Evidence and status

Collect runtime, freshness, recovery effort, and cost over consistent windows before claiming improvements. The full note links to official AWS guidance. No resources are provisioned and no performance results are asserted.

Read the full architecture note and AWS references on GitHub